

Lab 1: XOR, Autograd, and a Tiny Embedding Space

PyTorch tensors, a two-layer net on XOR, then cosine and a bias probe on constructed 2-D vectors.

Work in a **Jupyter** notebook with **PyTorch**. Do notes **1.1–1.4** first. You do not need a GPU.

When you are done: **File** → **Download as** → **HTML**, then upload the HTML on Canvas. Save the notebook before you export.

```
import numpy as np
import pandas as pd
import torch
from torch import nn
from pathlib import Path

# Put toy_embeddings.csv next to the notebook, or change this path.
EMB_PATH = Path("toy_embeddings.csv")
```

Download [toy_embeddings.csv](#) from the course site.

1. Tensors and autograd

1. Create `x = torch.tensor([0.5, -1.0, 2.0], requires_grad=True)`.
2. Compute $f = (x_1^2 + 2x_2^2 + 3x_3^2) / 2$ in PyTorch (0-based: `x[0]`, `x[1]`, `x[2]`).
3. Call `f.backward()`. Print `x.grad`. Write the gradient on paper first and check.
4. Why does this gradient match $\nabla f = (x_1, 4x_2, 6x_3)$? One sentence.

2. A two-layer net on XOR

XOR is the smallest problem that a linear model fails and an MLP can solve.

```
x = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y = torch.tensor([[0.0], [1.0], [1.0], [0.0]])
```

1. Fit a **linear** model `nn.Linear(2, 1)` with `BCEWithLogitsLoss` for 400 steps, `lr=0.5`. Print the predictions after a sigmoid. They should be stuck near 0.5. Count parameters first: this model has **3**.

`BCEWithLogitsLoss` is applied to the raw output z (logits), not to `sigmoid(z)`. If you wrap the linear layer in a sigmoid and then use this loss, you squash twice.

2. Fit an MLP: `Linear(2, 8) → ReLU → Linear(8, 1)`. Same loss, 2000 steps, `lr=0.1`. Print predictions. They should be near 0, 1, 1, 0. This model has **33** parameters: $2 \cdot 8 + 8$ plus $8 \cdot 1 + 1$.
 3. Plot the 2-D decision boundary of the MLP (a coarse grid is enough). Mark the four XOR points.
 4. In two sentences: why did the linear model fail, and what did ReLU add? Optional: on paper, check Goodfellow's two-ReLU XOR table from note **1.2** for $x = (1, 1)$.
-

3. Geometry of a tiny embedding space

The CSV is a **constructed** 2-D space so the pictures in note **1.4** are numbers you can compute. It is not a trained Word2Vec model.

```
df = pd.read_csv(EMB_PATH)
vecs = {row.word: np.array([row.x, row.y], dtype=float) for row in df.itertuples()}

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

1. Print `cosine(cat, dog)` and `cosine(cat, movie)`. Which pair is closer, and does that match the picture in the notes?
 2. Compute `king - man + woman`. Which word in the table (other than king/man/woman) is nearest in cosine? Repeat for `france` with `paris - france + italy`.
 3. **Intrinsic vs extrinsic**. This CSV has no downstream task. Invent one extrinsic test you *would* run if these were 300-D GloVe vectors for a news classifier. One paragraph.
 4. **Bias probe**. Compute the offset `he - she`. Project `engineer` and `nurse` onto that direction (scalar `dot(v, offset) / ||offset||`). Which occupation sits closer to `he`? This space was built to make the stereotype visible. In a real project you would report the same probe on the model you actually ship.
 5. Scatter-plot every word. Draw the king–queen parallelogram (four points and the two royal edges).
-

4. Project seed (ungraded this week, required later)

Write five lines at the bottom of the notebook:

- domain (text, vision+text, audio, retrieval, safety, ...)
- a question you can measure
- a public dataset you might use
- a baseline
- one risk (bias, leakage, compute, or data license)

You can change every line by the proposal date. The point is to start.